

SRI SAI RAM INSTITUTE OF TECHNOLOGY

Sai Leo Nagar, West Tambaram, Chennai – 44.



20CSPL601-ARTIFICIAL INTELLIGENCE LABORATORY LAB MANUAL (2023-2024)

III YEAR/VI SEMESTER

**DEPARTMENT OF COMPUTER AND
COMMUNICATION ENGINEERING**

VISION OF THE INSTITUTE

To be identified as a “Centre of Excellence” with high standards of Knowledge Dissemination and Research opportunities and to transform the students to imbibe qualities of technical expertise of international standards and high levels of ethical values, who in turn shall contribute to the advancement of society and human kind.

MISSION OF THE INSTITUTE

We shall dedicate and commit ourselves to attain and maintain excellence in Technical Education through commitment and continuous improvement of infrastructure and equipment and provide an inspiring environment for Learning, Research and Innovation for our students to transform them into complete human beings with ethical and social values.

VISION OF THE DEPARTMENT

To produce globally competent and socially responsible engineers who can address the engineering challenges and excel at an international level, in the advancement of Computer and Communication Engineering through research and academia

MISSION OF THE DEPARTMENT

Mission 1: To provide a good environment with latest technological infrastructure facilities, teaching learning ambience and interaction with industry in the area of Computer and Communication Engineering.

Mission 2: To inculcate graduates with high social responsibility, right attitude, discipline and an inclination towards offering their professional expertise in serving the society

PROGRAMME EDUCATIONAL OBJECTIVES

PEO1:-Impart strong foundation in basic sciences, mathematics and engineering fundamentals, knowledge and capability.

PEO2:-Apply design principles and best practices for developing quality products for scientific and business applications.

PEO3:-Inculcate high professionalism among the students by providing technical and soft skills with ethical standards.

PEO4:-Promote collaborative learning and spirit of team work through multidisciplinary projects and diverse professional activities.

PEO5:-Indoctrinate an attitude in the graduates for a life- long learning process.

PROGRAM OUTCOMES (PO's)

Engineering Graduates will be able to:

1. Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

2. Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

3. Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

4. Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

5. Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

6. The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

7. Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

8. Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

9. Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

10. Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

11. Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

12. Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change

PROGRAMME SPECIFIC OUTCOMES

PSO1:-Apply appropriate technology for the implementation of modern communication systems

PSO2:-Develop quality software for scientific and business applications by applying software engineering principles and practices.

OBJECTIVES:

The student should be made:

- To learn Prolog Program.
- To implement in prolog and its working environment
- To Implement N-Queen problem and puzzle problem using Prolog
- To analyze the problem using BFS and DFS algorithm

LIST OF EXPERIMENTS:

1. Study of Prolog
2. Write simple fact for the statements using prolog
3. Write predicates one converts centigrade temperature to Fahrenheit, other checks if a temperature is below freezing.
4. Write a program to solve 4-Queen problem.
5. Write a program to solve 8-puzzle problem.
6. Write a program to solve any problem using Breadth First Search.
7. Write a program to solve any problem Depth First Search
8. Write a program to solve Travelling salesman Problem
9. Write a prolog program to solve water jug problem.
10. Write a program to solve missionaries and cannibal problem
11. Write a prolog program to implement library management system.

TOTAL: 45 PERIODS

COURSE OUTCOMES:

At the end of the course, the student should be able to:

1. Interpret the concepts of Turbo and Prolog programming in AI. (K3)
2. Examine First order predicate logic to solve AI problems. (K4)
3. Apply Informed search strategies to solve AI problems. (K3)
4. Apply Uninformed search strategies to solve AI problems. (K3)
5. Select State Space Searching method to solve AI problems. (K3)
6. Demonstrate an application using Natural Language Processing. (K3)

CO,PO,PSO MAPPING:

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2
CO1	2	1	2	-	1	-	1	-	-	-	-	-	1	2
CO2	3	2	2	1	1	-	-	-	-	-	-	-	2	2
CO3	3	2	1	-	2	-	-	-	-	-	-	1	2	1
CO4	3	2	1	-	2	-	-	-	-	-	-	1	2	1
CO5	2	2	2	-	1	-	1	-	-	-	-	-	2	2
CO6	2	2	1	2	2	-	-	-	-	-	-	-	2	1

INDEX

EX.NO	NAME OF THE EXPERIMENT	PAGE.NO
1a	STUDY OF PROLOG - BASICS	8
1b	STUDY OF PROLOG – EXTENDED	13
2	SIMPLE FACT FOR THE STATEMENTS	19
3	TEMPERATURE CONVERSION	25
4	FACTORIAL OF A NUMBER	26
5	GCD OF TWO NUMBERS	27
6	N-QUEEN PROBLEM	28
7	WATER JUG PROBLEM	31
8	DEPTH FIRST SEARCH	36
9	BREADTH FIRST SEARCH	38
10	MISSIONARIES AND CANNIBAL PROBLEM	41
11	TRAVELLING SALESMAN PROBLEM	46
12	LIBRARY MANAGEMENT SYSTEM	49
CONTENT BEYOND SYLLABUS		
13	TOWERS OF HANOI	53
14	CRYPT ARITHMETIC PROBLEM	55

STUDY OF PROLOG - BASICS

AIM:

To Study about Prolog basic representation

Prolog:

Prolog or PROgramming in LOGics is a logical and declarative programming language. It is one major example of the fourth generation language that supports the declarative programming paradigm. This is particularly suitable for programs that involve symbolic or non-numeric computation. This is the main reason to use Prolog as the programming language in Artificial Intelligence, where symbol manipulation and inference manipulation are the fundamental tasks. In Prolog, we need not mention the way how one problem can be solved, we just need to mention what the problem is, so that Prolog automatically solves it. However, in Prolog we are supposed to give clues as the solution method.

Prolog language basically has three different elements –

Facts – The fact is predicate that is true, for example, if we say, “Tom is the son of Jack”, then this is a fact.

Rules – Rules are extensions of facts that contain conditional clauses. To satisfy a rule these conditions should be met. For example, if we define a rule as –

grandfather(X, Y) :- father(X, Z), parent(Z, Y)

This implies that for X to be the grandfather of Y, Z should be a parent of Y and X should be father of Z.

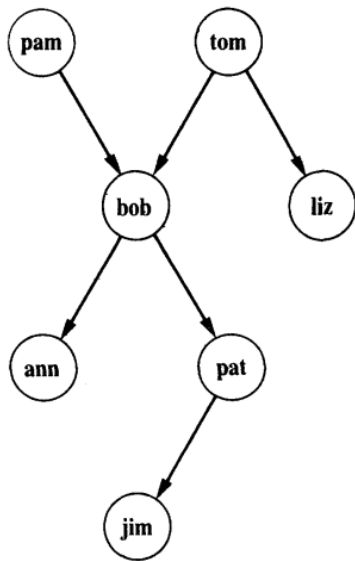
Applications of Prolog:

- Intelligent Database Retrieval
- Natural Language Understanding
- Specification Language
- Machine Learning
- Robot Planning

- Automation System
- Problem Solving

STUDY:

symbolic, non-numeric computation. suited for solving problems that involve objects and relations between objects



Tom is a parent of Bob can be written in Prolog as:

`parent(tom, bob).`

parent as the name of a relation; tom and bob are its arguments.

The whole Family Tree

```
//one.pl  
  
parent(pam, bob).  
parent(tom, bob).  
parent(tom, liz).  
parent(bob, ann).  
parent(bob, pat).  
parent(pat, jim).
```

This program consists of six clauses.

Prolog can be posed some questions about the parent relation

OUTPUT:

?- parent(bob,pat).

true.

?- parent(liz,pat).

false.

?- parent(tom,ben).

false.

?- parent(X,liz).

X = tom.

?- parent(bob, X).

X = ann .

?- parent(bob, X).

X = ann ;

X = pat.

?- parent(X, Y).

X = pam,

Y = bob ;

X = tom,

Y = bob ;

X = tom,

Y = liz ;

X = bob,

Y = ann ;

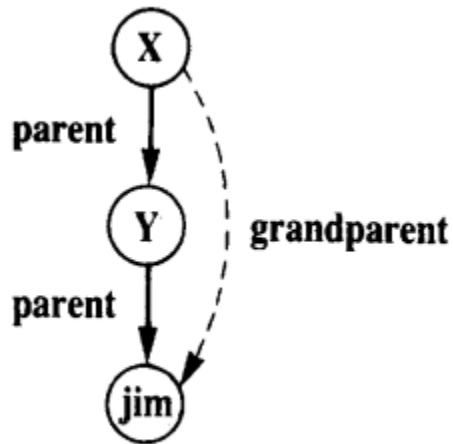
X = bob,

Y = pat ;

X = pat,

Y = jim.

The GRANDPARENT relation expressed as a composition of two parent relations.



(1) Who is a parent of Jim? Assume that this is some Y.

(2) Who is a parent of Y? Assume that this is some X.

?- parent(Y, jim), parent(X, Y).

Y = pat,

X = bob

?- parent(tom, X), parent(X, Y).

X = bob,

Y = ann ;

X = bob,

Y = pat ;

Adding grandparent relation to the existing program

grandparent(X,Y):-parent(X,Z),parent(Z,Y).

?- grandparent(pam,pat).

true.

?- grandparent(pam,ann).

true.

?- grandparent(tom,ann).

true .

EXERCISES

1. parent(jim,X)

2. parent(X,jim)

3. parent(pam,X),parent(X,pat)

4. parent(pam,X),parent(X,Y),parent(Y,jim)

5. Who is Pat's parent?

6. Does Liz have a child?

7. Who is Pat's grandparent?

RESULT: Thus, Basics of Prolog studied successfully.

STUDY OF PROLOG - EXTENDED

AIM:

To Study about Prolog advanced concepts like structures and matching

STUDY:

- Prolog programs can be extended by simply adding new clauses.
- Prolog clauses are of three types: facts, rules and questions.
- Facts declare things that are always, unconditionally true.
- Rules declare things that are true depending on a given condition.
- By means of questions the user can ask the program what things are true.
Prolog clauses consist of the head and the body. The body is a list of goals separated by commas. Commas are understood as conjunctions
- A variable can be substituted by another object. we say that a variable becomes instantiated.
- Variables are assumed to be universally quantified and are read as “for all”

EXTENDING THE EXAMPLES: (Ex.No:1a)

female(pam).

male(tom).

male(bob).

female(liz).

female(pat).

female(ann).

male(jim).

The relations introduced here are MALE and FEMALE. These relations are unary relations. unary relations can be used to declare simple yes/no properties of objects

OUTPUT:

?- male(jim).

true.

?- female(pam).

true.

?- female(liz).

true.

?- male(X).

X = tom ;

X = bob ;

X = jim.

let us introduce the CHILDREN relation as the inverse of the parent relation.

For all X and Y, Y is a child of X if X is a parent of Y.

child(Y, X) :- parent(X, Y).

Output:

?- child(liz,tom).

true.

?- child(X,tom).

X = bob ;

X = liz.

Rules specify things that may be true if some condition is satisfied.

Rules have

a condition part (the right-hand side of the rule)(body) and

a conclusion part (the left-hand side of the rule). (head)

Let us Introduce MOTHER relation

For all X and Y.

X is the mother of Y

if X is a parent of Y and X is a female.

mother(X, Y) :- parent(X, Y), female(X).

OUTPUT:

?- mother(pam,bob).

true .

?- mother(X,bob).

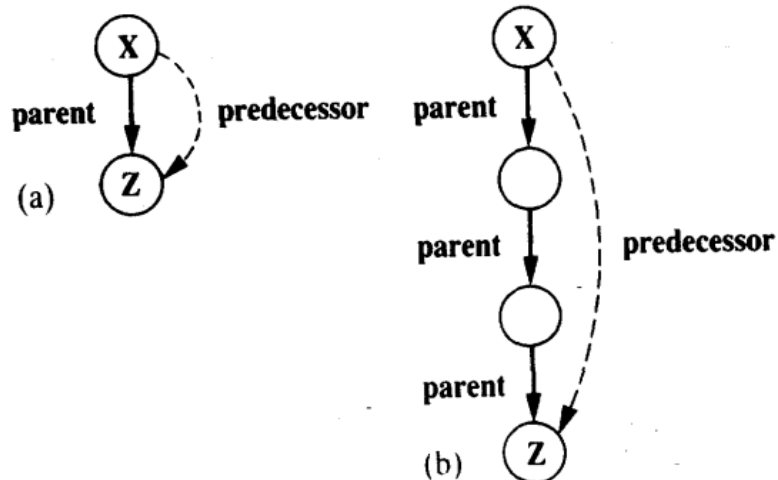
X = pam ;

false.

?- mother(X,Y).

X = pam,

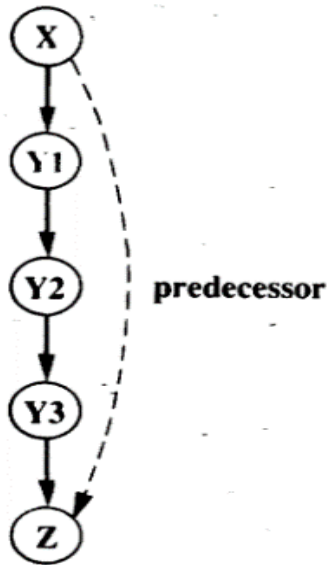
A RECURSIVE RULE DEFINITION



(a) X is a direct predecessor of Z; (b) X is an indirect predecessor of Z.

For all X and Z, X is a predecessor of Z if X is a parent of Z.

predecessor(X, Z) :- parent(X, Z).



`predecessor(X, Z) :- parent( X, Y1), parent( Y1, Y2), parent( Y2, Y3), parent( Y3, Z).`

This program is lengthy and, more importantly, it only works to some extent

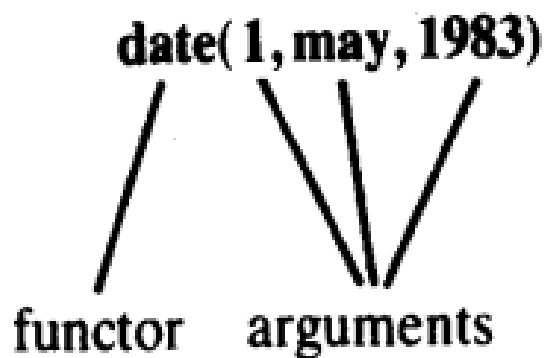
Recursive Definitions

`predecessor(X, Z) :- parent( X, Z).`

`predecessor(X, Z) :- parent( X, Y), predecessor(Y, Z).`

STRUCTURES: Structured objects (or simply structures) are objects that have several components.

`date( 1, may, 1983)`



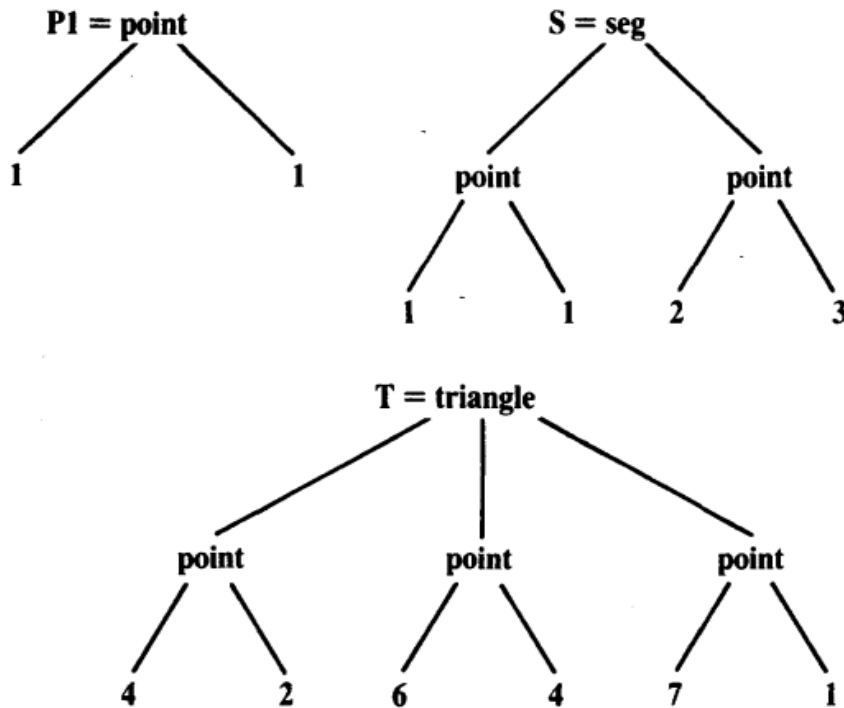
`date(Day, may, 1983)` – Any Day in May

`p1 : point(l,l)`

p2: point(2,3)

S : seg(P1, P2): seg(point(1,1), point(2,3))

T : triangle(point(4,Z), point(6,4), point(7,l))



MATCHING:

Given two terms, we say that they match if. they are identical, or the variables in both terms can be instantiated to objects in such a way that after the substitution of variables by these objects the terms become identical.

Date Example:

?- date(D1,M1,1983)=date(D,M,Y1).

D1 = D,

M1 = M,

Y1 = 1983.

?- date(D1,M1,1983)=date(D,M,1987).

false.

?- date(D,M,1983)=date(D1,may,Y1),date(D,M,1983)=date(15,M,Y).

D = D1, D1 = 15,

M = may,

Y1 = Y, Y = 1983.

Point Example

vertical(seg(point(X,Y),point(X,Y1))).

horizontal(seg(point(X,Y),point(X1,Y))).

Output:

?- vertical(seg(point(1,4),point(1,5))).

true.

?- horizontal(seg(point(2,10),point(10,10))).

true.

?- horizontal(seg(point(1,5),point(5,Y))).

Y = 5.

?- vertical(seg(point(2,3), P)).

P = point(2, _).

?- vertical(S),horizontal(S).

S = seg(point(_A, _B), point(_A, _B))

RESULT: Thus the Prolog concepts like recursive function. structures, matching was studied successfully

EX.NO: 2

SIMPLE FACT FOR THE STATEMENTS

AIM:

To write simple fact for the statements using prolog

REPRESENTATION IN PROLOG:

Prolog expressions are comprised of the following truth-functional symbols, which have the same interpretation as in the predicate calculus.

English	Predicate Calculus	PROLOG
and	$\wedge$	,
or	$\vee$	;
if	$-->$	:-
not	$\sim$	not

VARIABLES AND NAMES:

Variables begin with an uppercase letter. Predicate names, function names, and the names for objects must begin with a lowercase letter. Rules for forming names are the same as for the predicate calculus.

mother_of

male

female

greater_than

Socrates

FACTS: A fact is a predicate expression that makes a declarative statement about the problem domain. Whenever a variable occurs in a Prolog expression, it is assumed to be universally quantified. Note that all Prolog sentences must end with a period.

likes (John, Susie). /* John likes Susie */

likes (X, Susie). /* Everyone likes Susie */
 likes (John, Y). /* John likes everybody */
 likes (John, Y), likes (Y, John). /* John likes everybody and everybody likes John */
 likes (John, Susie); likes(John, Mary). /* John likes Susie or John likes Mary */
 not (likes (John, Pizza)). /* John does not like pizza */
 likes (John, Susie):- likes (John, Mary)./* John likes Susie if John likes Mary.

RULES:

A rule is a predicate expression that uses logical implication (:-) to describe a relationship among facts. Thus, a Prolog rule takes the form

left_hand_side :- right_hand_side .

This sentence is interpreted as: left_hand_side if right_hand_side. The left_hand_side is restricted to a single, positive, literal, which means it must consist of a positive atomic expression. It cannot be negated and it cannot contain logical connectives.

This notation is known as a Horn clause. In Horn clause logic, the left-hand side of the clause is the conclusion, and must be a single positive literal. The right-hand side contains the premises. The Horn clause calculus is equivalent to the first-order predicate calculus.

EXAMPLES OF VALID RULES:

friends(X,Y) :- likes(X,Y),likes(Y,X). /* X and Y are friends if they like each other */
 hates(X,Y) :- not(likes(X,Y)). /* X hates Y if X does not like Y. */
 enemies(X,Y) :- not(likes(X,Y)),not(likes(Y,X)). /* X and Y are enemies if they don't like each other */

EXAMPLES OF INVALID RULES:

left_of(X,Y) :- right_of(Y,X) /* Missing a period */
 likes(X,Y),likes(Y,X) :- friends(X,Y). /* LHS is not a single literal */
 not(likes(X,Y)) :- hates(X,Y). /* LHS cannot be negated */

QUERIES:

The Prolog interpreter responds to queries about the facts and rules represented in its database. The database is assumed to represent what is true about a particular problem domain. In making a query you are asking Prolog whether it can prove that your query is true. If so, it answers "yes" and displays any variable bindings that it made in coming up with the answer. If it fails to prove the query true, it answers "No".

Whenever you run the Prolog interpreter, it will prompt you with ?-. For example, suppose our database consists of the following facts about a fictitious family.

father_of(joe,paul).

father_of(joe,mary).

mother_of(jane,paul).

mother_of(jane,mary).

male(paul).

male(joe).

female(mary).

female(jane).

Closed World Assumption:

The Prolog interpreter assumes that the database is a closed world -- that is, if it cannot prove something is true, it assumes that it is false. This is also known as negation as failure -- that is, something is false if PROLOG cannot prove it true given the facts and rules in its database.

EXAMPLE 2:

Statements:

1. The Cakes are delicious.
2. The Pickles are delicious.
3. The Pickles are spicy.
4. Priya relishes coffee.
5. Priya likes food if they are delicious.
6. Prakash likes food if they are spicy and delicious.

Statements in Prolog:

1. delicious(cakes).
2. delicious(pickles).
3. spicy(pickles).
4. relishes(priya, coffee).
5. likes(priya, Food) if delicious(Food). %Here Food is a variable
6. likes(prakash,Food) if spicy(Food) and delicious(Food).

Program Clauses:

%EX2.pl

```
delicious(cakes).  
delicious(pickles).  
spicy(pickles).  
relishes(priya, coffee).  
% here Food is a variable  
likes(priya, Food):-delicious(Food).  
likes(prakash, Food):- spicy(Food), delicious(Food).
```

OUTPUT:

```
?- delicious(pickles).  
true.  
?- spicy(pickles).  
true.  
?- relishes(priya,coffee).  
true.  
?- delicious(Food).  
Food = cakes ;  
Food = pickles.
```

Result: Thus, simple facts are represented by using prolog

NOTE:**OPERATORS IN PROLOG**

+	Addition
-	Subtraction
*	Multiplication
/	Division
Mod	modulo, the remainder of integer division
$X > Y$	X greater than Y
$X < Y$	X less than Y
$X \geq Y$	X is greater than or equal to Y
$X \leq Y$	X is less than or equal to Y
$X = Y$	the values of X and Y are equal
$X \neq Y$	the values of X and Y are not equal

?- X is 1+2.

X = 3

?- X is 3/2.

X = 1.5.

?- Y is 3 div 2.

Y = 1.

?- 10*5 > 45.

true.

?- 1+2 =:= 2+1.

true.

?- 1+2 \= 2+2.

true.

?- 1+A=B+2.

A = 2,

B = 1.

EX.NO: 3

TEMPERATURE CONVERSION

AIM:

To write predicates one converts centigrade temperature to Fahrenheit, other checks if a temperature is below freezing.

Formula for Centigrade (C) temperatures to Fahrenheit (F) -

$$F = C * 9 / 5 + 32$$

PROGRAM

%ex3.pl

%ctof converts C to F

ctof(C,F):-F is C*9/5+32.

%freezing function check the temperature

freezing(F):-F=<32.

OUTPUT:

?- ctof(100,F).

F = 212.

?- freezing(20).

true.

?- freezing(40).

false.

Result: Thus, prolog program to convert centigrade temperature to Fahrenheit is executed successfully

EX.NO: 4

FACTORIAL OF A NUMBER

AIM:

To write prolog program to find Factorial of a Given number.

PROGRAM

%ex4.pl

```
fact(0,1).  
fact(N,F):-N>0,N1 is N-1,fact(N1,F1),F is N * F1.
```

OUTPUT:

?- fact(5,A).

A = 120 .

?- fact(10,B).

B = 3628800

?- fact(5,720).

false.

Result: Thus, prolog program to find factorial of a number is executed successfully

EX.NO: 5

GCD OF TWO NUMBERS

AIM:

To write prolog program to find GCD of two Numbers.

PROGRAM

%ex5.pl

```
gcd(X,X,X).  
gcd(X,Y,D):- X < Y,Y1 is Y-X,gcd(X,Y1,D).  
gcd(X,Y,D):- Y < X,X1 is X-Y,gcd(X1,Y,D).
```

OUTPUT:

?- gcd(75,30,B).

B = 15 .

?- gcd(15,60,A).

A = 15 .

?- gcd(75,30,B).

B = 15 .

Result: Thus, prolog program to find GCD of two Numbers is executed Successfully

EX.NO: 6

N-QUEEN PROBLEM

AIM:

To Write a Prolog program to solve N-Queens Problem

PROCEDURE:

- * The problem here is to place eight queens on the empty chessboard in such a way that no queen attacks any other queen.
- * Represent the position by a list of eight items, each of them corresponding to one queen.
- * Each item in the list will specify a square of the board on which the corresponding queen is sitting.
- * Example: 1/4 → Queen 1 in 4th Position
- * Having chosen this representation, the problem is to find such a list of the form $IX_1/Y_1, X_2/Y_2, X_3/Y_3, \dots, X_8/Y_8$ which satisfies the no-attack requirement.

The solution relation can then be formulated by considering two cases

Case 1: The list of queens is empty: the empty because there is no attack.

Case 2: The list of queens is non-empty: then it looks like this: $[X/Y \mid \text{Others}]$

In case 2, the first queen is at some square X/Y and the other queens are at squares specified by the list others. If this is to be a solution then the following conditions must hold:

There must be no attack between the queens in the list others; that is, Others itself must also be a solution.

X and Y must be integers between 1 and 8.

A queen at square X/Y must not attack any of the queens in the list Others

noattack relation:

(1) If the list Qlist is empty then the relation is certainly true because there is no queen to be attacked.

(2) If Qlist is not empty then it has the form [Q1|Qlist1] and two conditions must be satisfied:

(a) the queen at Q must not attack the queen at Q1, and

(b) the queen at Q must not attack any of the queens in Qlist1

To specify that a queen at some square does not attack another square is easy:

The two squares must not be in the same row, the same column or the same diagonal

our solution template guarantees that all the queens are in different columns

* The Y-coordinates of the queens are different, and they are not in the same.

Diagonal, either upward or downward; that is, the distance between the squares in the X-direction must not be equal to that in the Y-direction

PROGRAM:

```
solution([]).  
solution([X/Y | Others]):-  
    solution(Others),  
    mem(Y,[1,2,3,4,5,6,7,8]),  
    noattack(X/Y,Others).  
noattack(_,[]).  
noattack(X/Y,[X1/Y1 | Others]):-  
    Y \= Y1,  
    Y1-Y \= X1-X,  
    Y1-Y \= X-X1,  
    noattack(X/Y,Others).  
mem(X,[X|_]).  
mem(X,[_|T]):-mem(X,T).  
template([1/Y1,2/Y2,3/Y3,4/Y4,5/Y5,6/Y6,7/Y7,8/Y8]).
```

OUTPUT:

?- template(S),solution(S).

S = [1/4, 2/2, 3/7, 4/3, 5/6, 6/8, 7/5, 8/1] ;

S = [1/5, 2/2, 3/4, 4/7, 5/3, 6/8, 7/6, 8/1]

RESULT: Thus, N-Queen's Problem solved by using prolog program.

EX.NO: 7

WATER JUG PROBLEM

AIM:

To Write a Prolog program to solve Water Jug Problem

PROCEDURE:

You are given two jugs, a 4-gallon one and a 3-gallon one. Neither has any measuring mark on it. There is a pump that can be used to fill the jugs with water. Get exactly 2 gallons of water into the 4-gallon jug.

The state space for this problem can be described as the set of ordered pairs of integers **(x,y)**

Where,

X represents the quantity of water in the 4-gallon jug **X= 0,1,2,3,4**

Y represents the quantity of water in 3-gallon jug **Y=0,1,2,3**

Start State: (0,0)

Goal State: (2,0)

Generate production rules for the water jug problem

Production Rules:

Rule	State	Process
1	$(X,Y \mid X < 4)$	$(4,Y)$ {Fill 4-gallon jug}
2	$(X,Y \mid Y < 3)$	$(X,3)$ {Fill 3-gallon jug}
3	$(X,Y \mid X > 0)$	$(0,Y)$ {Empty 4-gallon jug}
4	$(X,Y \mid Y > 0)$	$(X,0)$ {Empty 3-gallon jug}
5	$(X,Y \mid X+Y \geq 4, Y > 0)$	$(4,Y-(4-X))$ {Pour water from 3-gallon jug into 4-gallon jug until 4-gallon jug is full}

6	$(X,Y \mid X+Y \geq 3, X > 0)$	$(X-(3-Y),3)$ {Pour water from 4-gallon jug into 3-gallon jug until 3-gallon jug is full}
7	$(X,Y \mid X+Y \leq 4, Y > 0)$	$(X+Y,0)$ {Pour all water from 3-gallon jug into 4-gallon jug}
8	$(X,Y \mid X+Y \leq 3 \wedge X > 0)$	$(0,X+Y)$ {Pour all water from 4-gallon jug into 3-gallon jug}
9	$(0,2)$	$(2,0)$ {Pour 2 gallon water from 3 gallon jug into 4 gallon jug}

PROGRAM:

fill(x,y).

fill(2,0):-

nl,

write('Goal State is Reached....').

fill(X,Y):-

X=0,

Y=<1,

nl,

write('Fill the 4-Gallon Jug:(4,)',write(Y),write(')'),fill(4,Y).

fill(X,Y):-

Y=0,

X>=3,

nl,

write('Fill the 3-Gallon Jug:('),

write(X),

write(',3)'),


```
fill(X,3).
```

```
fill(X,Y):-
```

```
    X+Y>=4,
```

```
    Y=3,
```

```
    X=3,
```

```
    Y1 is Y-(4-X),
```

```
    nl,
```

```
    write('Pour water from 3-Gallon Jug to 4-Gallon until is full:(4, '),
```

```
    write(Y1),
```

```
    write(')'),
```

```
    fill(4,Y1).
```

```
fill(X,Y):-
```

```
    X+Y>=3,
```

```
    X=4,
```

```
    Y=<1,
```

```
    X1 is X-(3-Y),
```

```
    nl,
```

```
    write('Pour water from 4-Gallon Jug to 3-Gallon until is full: '),
```

```
    write(X1),
```

```
    write(',3)'),
```

```
    fill(X1,3).
```

```
fill(X,Y):-
```

```
    X+Y=<4,
```

```
    X=0,
```

```
    Y>1,
```

X1 is X+Y,

nl,

write('pour all the water from 3-Gallon Jug to 4-Gallon:('),

write(X1),

write('0)'),

fill(X1,0).

fill(X,Y):-

X+Y<3,

Y=0,

Y1 is X+Y,

nl,

write('Pour all the water from 4-Gallon Jug too 3-Gallon:(0,') ,

write(Y1),

write(')'),

fill(0,Y).

fill(X,Y):-

Y>=2,

X=4,

nl,

write('Empty the 4-Gallon Jug on Ground:(0,') ,

write(Y),

write(')'),

fill(0,Y).

fill(X,Y):-

Y=3,

```

X>=1,
nl,
write('Empty the 3-Gallon Jug on Ground:('),
write(X),
write(',0)'),
fill(X,0).

fill(X,Y):- X>4,Y<3,
    write('4L Jug Overflowed. '),nl.

fill(X,Y):- X<4, Y>3,
    write('3L Jug Overflowed. '),nl.

fill(X,Y):-X>4,Y>3,
    write('4L3L Jug Overflowed. '),nl.

```

OUTPUT:

```

?- fill(4,0).
Fill the 3-Gallon Jug:(4,3)
Empty the 4-Gallon Jug on Ground:(0,3)
pour all the water from 3-Gallon Jug to 4-Gallon:(3,0)
Fill the 3-Gallon Jug:(3,3)
Pour water from 3-Gallon Jug to 4-Gallon until is full:(4,2)
Empty the 4-Gallon Jug on Ground:(0,2)
pour all the water from 3-Gallon Jug to 4-Gallon:(2,0)
Goal State is Reached....

```

RESULT: Thus, the Water Jug problem Solved by using Prolog

EX.NO: 8

DEPTH FIRST SEARCH

AIM:

To write a prolog program to implement Depth First Search.

PROCEDURE:

- Depth-first search is an algorithm used for traversing or searching a tree or graph data structure.
- The algorithm starts at the root node and explores as far as possible along each branch before backtracking.
- It works by maintaining a stack of nodes to visit, starting with the root node.
- At each step, the algorithm pops the top node from the stack and visits its unvisited neighbors, pushing them onto the stack.
- This process continues until the stack is empty or the goal node is found.

PROGRAM:

s(a,b).

s(a,c).

s(b,d).

s(b,e).

s(c,f).

s(c,g).

s(d,h).

s(e,i).

s(e,j).

s(f,k).

goal(f).

```

goal(j).
mem(X,[X|_]).
mem(X,[_|Tail]):-mem(X,Tail).
solve(Node,Solution):-
    dfs([],Node,Solution).
dfs(Path,Node,[Node|Path]):-
    goal(Node).
dfs(Path,Node,Sol):-
    s(Node,Node1),
    not(mem(Node1,Path)),
    dfs([Node|Path],Node1,Sol).

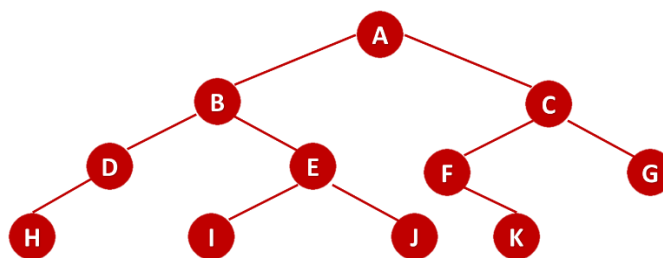
```

OUTPUT:

?- solve(a,S).

S = [j, e, b, a] ;

S = [f, c, a]



RESULT: Thus, the prolog program to implement DFS was executed Successfully.

EX.NO: 9

BREADTH FIRST SEARCH

AIM:

To write a prolog program to implement Breadth First Search.

PROCEDURE:

Breadth-First Search (BFS) is a graph traversal algorithm that explores all the vertices of a graph in breadth-first order, i.e., it visits all the nodes at the same depth before moving on to the nodes at the next depth.

PROGRAM:

s(a,b).

s(a,c).

s(b,d).

s(b,e).

s(c,f).

s(c,g).

s(d,h).

s(e,i).

s(e,j).

s(f,k).

goal(f).

goal(j).

solve(Start,Solution):-

 bfs([[Start]],Solution).

bfs([[Node|Path]|_],[Node|Path]]):-

```

    goal(Node).

bfs([Path|Paths],Solution):-
    extend(Path,NewPaths),
    write(NewPaths),
    nl,
    conc(Paths,NewPaths,Paths1),bfs(Paths1,Solution).

extend([Node|Path],NewPaths):-

    bagof([NewNode,Node|Path],(s(Node,NewNode),not(member(NewNode,[Node|Path]))
    ),NewPaths),!.

extend(_,_).

conc([],L,L).

conc([X|L1],L2,[X|L3]):-nl,write('conc'),write(X),write('
'),write(L1),write(L2),conc(L1,L2,L3).

```

OUTPUT:

```

?- solve(a,S).

[[b,a],[c,a]]
[[d,b,a],[e,b,a]]
conc[c,a] [[d,b,a],[e,b,a]][[f,c,a],[g,c,a]]
conc[d,b,a] [[e,b,a]][[f,c,a],[g,c,a]]
conc[e,b,a] [[f,c,a],[g,c,a]][[h,d,b,a]]
conc[e,b,a] [[f,c,a],[g,c,a]][[h,d,b,a]]
conc[f,c,a] [[g,c,a]][[h,d,b,a]]
conc[g,c,a] [[h,d,b,a]][[i,e,b,a],[j,e,b,a]]
conc[f,c,a] [[g,c,a],[h,d,b,a]][[i,e,b,a],[j,e,b,a]]

```

```
conc[g,c,a] [[h,d,b,a]][[i,e,b,a],[j,e,b,a]]
```

```
conc[h,d,b,a] [] [[i,e,b,a],[j,e,b,a]]
```

```
S = [f, c, a] ;
```

Result: Thus, the prolog program to implement BFS was executed Successfully.

EX.NO: 10

MISSIONARIES AND CANNIBAL PROBLEM

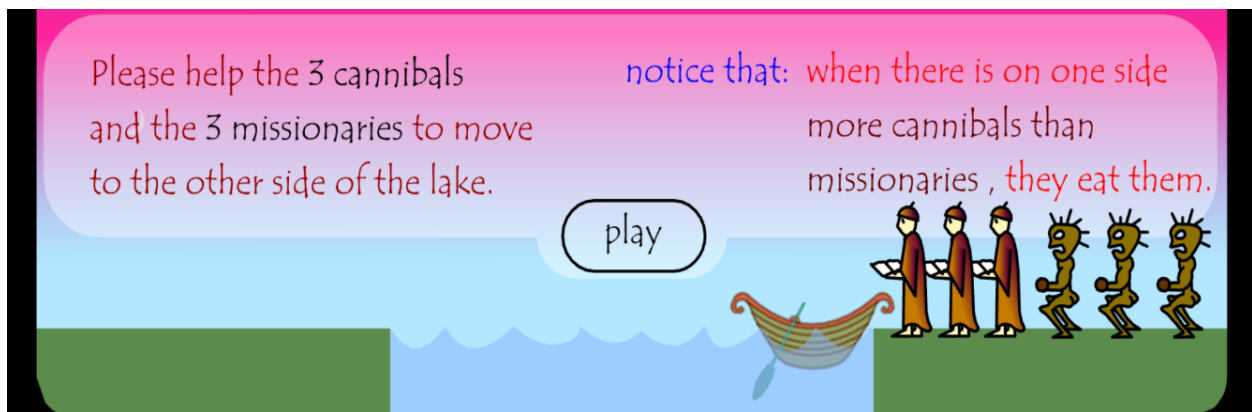
AIM:

To write a prolog program to solve Missionaries and Cannibal Problem.

PROCEDURE:

The Missionaries and Cannibals problem is a classic problem in Artificial Intelligence that involves three missionaries and three cannibals on one side of a river, along with a boat that can carry at most two people at a time.

The goal is to move all the people to the other side of the river without ever leaving more missionaries than cannibals on either side, or else the cannibals will eat the missionaries



PROGRAM:

% Represent a state as [CL,ML,B,CR,MR]

start([3,3,left,0,0]).

goal([0,0,right,3,3]).

legal(CL,ML,CR,MR) :-

% is this state a legal one?

$ML \geq 0$, $CL \geq 0$, $MR \geq 0$, $CR \geq 0$,

$(ML \geq CL ; ML = 0)$,

$(MR \geq CR ; MR = 0)$.

% Possible moves:

move([CL,ML,left,CR,MR],[CL,ML2,right,CR,MR2]):-

% Two missionaries cross left to right.

MR2 is $MR+2$,

ML2 is $ML-2$,

legal(CL,ML2,CR,MR2).

move([CL,ML,left,CR,MR],[CL2,ML,right,CR2,MR]):-

% Two cannibals cross left to right.

CR2 is $CR+2$,

CL2 is $CL-2$,

legal(CL2,ML,CR2,MR).

move([CL,ML,left,CR,MR],[CL2,ML2,right,CR2,MR2]):-

% One missionary and one cannibal cross left to right.

CR2 is $CR+1$,

CL2 is $CL-1$,

MR2 is $MR+1$,

ML2 is $ML-1$,

legal(CL2,ML2,CR2,MR2).

move([CL,ML,left,CR,MR],[CL,ML2,right,CR,MR2]):-

% One missionary crosses left to right.

MR2 is $MR+1$,

```

    ML2 is ML-1,
    legal(CL,ML2,CR,MR2).
move([CL,ML,left,CR,MR],[CL2,ML,right,CR2,MR2]):-
    % One cannibal crosses left to right.
    CR2 is CR+1,
    CL2 is CL-1,
    legal(CL2,ML,CR2,MR2).
move([CL,ML,right,CR,MR],[CL,ML2,left,CR,MR2]):-
    % Two missionaries cross right to left.
    MR2 is MR-2,
    ML2 is ML+2,
    legal(CL,ML2,CR,MR2).
move([CL,ML,right,CR,MR],[CL2,ML,left,CR2,MR2]):-
    % Two cannibals cross right to left.
    CR2 is CR-2,
    CL2 is CL+2,
    legal(CL2,ML,CR2,MR2).
move([CL,ML,right,CR,MR],[CL2,ML2,left,CR2,MR2]):-
    % One missionary and one cannibal cross right to left.
    CR2 is CR-1,
    CL2 is CL+1,
    MR2 is MR-1,
    ML2 is ML+1,
    legal(CL2,ML2,CR2,MR2).
move([CL,ML,right,CR,MR],[CL,ML2,left,CR,MR2]):-

```

```

    % One missionary crosses right to left.
    MR2 is MR-1,
    ML2 is ML+1,
    legal(CL,ML2,CR,MR2).
move([CL,ML,right,CR,MR],[CL2,ML,left,CR2,MR2):-
    % One cannibal crosses right to left.
    CR2 is CR-1,
    CL2 is CL+1,
    legal(CL2,ML,CR2,MR).

% Recursive call to solve the problem
path([CL1,ML1,B1,CR1,MR1],[CL2,ML2,B2,CR2,MR2],Explored,MovesList):-
    move([CL1,ML1,B1,CR1,MR1],[CL3,ML3,B3,CR3,MR3]),
    not(member([CL3,ML3,B3,CR3,MR3],Explored)),

    path([CL3,ML3,B3,CR3,MR3],[CL2,ML2,B2,CR2,MR2],[[CL3,ML3,B3,CR3,MR3]|Explored],
    [ [ [CL3,ML3,B3,CR3,MR3],[CL1,ML1,B1,CR1,MR1]] | MovesList ]]).

% Solution found
path([CL,ML,B,CR,MR],[CL,ML,B,CR,MR],_,MovesList):-
    output(MovesList).

% Printing
output([]) :- nl.
output([[A,B]|MovesList]) :-
    output(MovesList),
    write(B), write(' -> '), write(A), nl.

% Find the solution for the missionaries and cannibals problem

```

find :-

path([3,3,left,0,0],[0,0,right,3,3],[[3,3,left,0,0]],_).

OUTPUT:

?- find.

[3,3,left,0,0] -> [1,3,right,2,0]

[1,3,right,2,0] -> [2,3,left,1,0]

[2,3,left,1,0] -> [0,3,right,3,0]

[0,3,right,3,0] -> [1,3,left,2,0]

[1,3,left,2,0] -> [1,1,right,2,2]

[1,1,right,2,2] -> [2,2,left,1,1]

[2,2,left,1,1] -> [2,0,right,1,3]

[2,0,right,1,3] -> [3,0,left,0,3]

[3,0,left,0,3] -> [1,0,right,2,3]

[1,0,right,2,3] -> [1,1,left,2,2]

[1,1,left,2,2] -> [0,0,right,3,3]

RESULT: Thus, the Missionaries and Cannibals problem solved by using Prolog

EX.NO: 11

TRAVELLING SALESMAN PROBLEM

AIM:

To Write a Prolog program to solve Travelling Salesman Problem.

PROCEDURE:

The Traveling Salesman Problem (TSP) is a classic optimization problem in which a salesman wants to visit a number of cities exactly once, and return to the starting city. The objective is to find the shortest possible route that visits every city.

PROGRAM:

```
edge(a, b, 3).  
edge(a, c, 5).  
edge(a, d, 7).  
edge(a, e, 8).  
edge(b, c, 9).  
edge(b, d, 10).  
edge(b, e, 3).  
edge(c, d, 5).  
edge(c, e, 8).  
edge(d, e, 6).  
edge(b, a, 3).  
edge(c, a, 4).  
edge(d, a, 2).  
edge(e, a, 7).  
edge(c, b, 4).
```

```
edge(d, b, 6).
```

```
edge(e, b, 3).
```

```
edge(d, c, 5).
```

```
edge(e, c, 8).
```

```
edge(e, d, 6).
```

```
edge(a, h, 2).
```

```
edge(h, d, 1).
```

```
/* Finds the length of a list, while there is something in the list it increments N when  
there is nothing left it returns.*/
```

```
len([], 0).
```

```
len([H|T], N):- len(T, X), N is X+1 .
```

```
/*Best path, is called by shortest_path. It sends it the paths found in a path, distance  
format*/
```

```
best_path(Visited, Total):- path(a, a, Visited, Total).
```

```
/*Path is expanded to take in distance so far and the nodes visited */
```

```
path(Start, Fin, Visited, Total) :- path(Start, Fin, [Start], Visited, 0, Total).
```

```
/*This adds the stopping location to the visited list, adds the distance and then calls  
recursive to the next stopping location along the path */
```

```
path(Start, Fin, CurrentLoc, Visited, Costn, Total) :-
```

```
    edge(Start, StopLoc, Distance), NewCostn is Costn + Distance, \+ member(StopLoc,  
CurrentLoc),
```

```
    path(StopLoc, Fin, [StopLoc|CurrentLoc], Visited, NewCostn, Total).
```

```
/*When we find a path back to the starting point, make that the total distance and  
make sure the graph has touch every node*/
```

```
path(Start, Fin, CurrentLoc, Visited, Costn, Total) :-
```

```
    edge(Start, Fin, Distance), reverse([Fin|CurrentLoc], Visited), len(Visited, Q),
```

(Q\=7 -> Total is 100000; Total is Costn + Distance).

/*This is called to find the shortest path, takes all the paths, collects them in holder.
Then calls pick on that holder which picks the shortest path and returns it*/

shortest_path(Path):-setof(Cost-Path, best_path(Path, Cost), Holder), pick(Holder, Path).

/* Is called, compares 2 distances. If cost is smaller than bcost, no need to go on. Cut it.*/

best(Cost-Holder, Bcost-_, Cost-Holder):- Cost<Bcost,!.

best(_, X, X).

/*Takes the top path and distance off of the holder and recursively calls it.*/

pick([Cost-Holder|R], X):- pick(R, Bcost-Bholder), best(Cost-Holder, Bcost-Bholder, X),!.

pick([X], X).

OUTPUT:

?- shortest_path(Path).

Path = 22-[a, h, d, c, e, b, a].

RESULT: Thus, the Travelling Salesperson Problem was solved by using Prolog

LIBRARY MANAGEMENT SYSTEM

AIM:

To write a Prolog program to simulate Library Management System.

PROGRAM:

% Define books with their properties

book(harry_potter_1, jk_rowling, 1997, fantasy).

book(harry_potter_2, jk_rowling, 1998, fantasy).

book(harry_potter_3, jk_rowling, 1999, fantasy).

book(the_hobbit, jrr_tolkien, 1937, fantasy).

book(the_lord_of_the_rings, jrr_tolkien, 1954, fantasy).

book(the_da_vinci_code, dan_brown, 2003, thriller).

book(angels_and_demons, dan_brown, 2000, thriller).

book(digital_fortress, dan_brown, 1998, thriller).

book(the_girl_with_the_dragon_tattoo, stieg_larsson, 2005, mystery).

book(the_girl_who_played_with_fire, stieg_larsson, 2006, mystery).

book(the_girl_who_kicked_the_hornets_nest, stieg_larsson, 2007, mystery).

% Define borrowers with their properties

borrower(john, doe, 12345).

borrower(jane, smith, 67890).

borrower(jack, black, 24680).

% Define the borrowed predicate

borrowed(harry_potter_1, 12345).

borrowed(the_hobbit, 67890).

borrowed(the_da_vinci_code, 24680).

borrowed(the_girl_with_the_dragon_tattoo, 12345).

borrowed(the_girl_who_played_with_fire, 67890).

% Define predicates for book search and borrower search

find_book_by_title(Title, Author, Year, Genre) :-

book(Title, Author, Year, Genre).

find_book_by_author(Author, Title, Year, Genre) :-

book(Title, Author, Year, Genre).

find_borrower_by_id(Id, FirstName, LastName) :-

borrower(FirstName, LastName, Id).

list_borrowed_books :-

borrowed(BookTitle, BorrowerId),

find_book_by_title(BookTitle, Author, Year, Genre),

find_borrower_by_id(BorrowerId, FirstName, LastName),

format('Book: ~w, Author: ~w, Year: ~w, Genre: ~w, Borrower: ~w ~w~n', [BookTitle, Author, Year, Genre, FirstName, LastName]).

OUTPUT:

?- find_book_by_author(jk_rowling,A,Y,G).

A = harry_potter_1,

Y = 1997,

G = fantasy .

?- find_borrower_by_id(12345,A,B).

A = john,

B = doe.

?- find_book_by_title(harry_potter_1,A,Y,G).

A = jk_rowling,

Y = 1997,

G = fantasy.

?- list_borrowed_books.

Book: harry_potter_1, Author: jk_rowling, Year: 1997, Genre: fantasy, Borrower: john
doe

true ;

Book: the_hobbit, Author: jrr_tolkien, Year: 1937, Genre: fantasy, Borrower: jane smith
true

RESULT: Thus, Library Management system is simulated using Prolog.

CONTENT BEYOND SYLLABUS

EX.NO: 13

TOWERS OF HANOI

AIM:

To Write a Prolog Program to solve Towers of Hanoi

PROGRAM:

% Prolog program to representation of 'Tower of Hanoi'

move(1,X,Y,_):-

 write('Move disk from '),

 write(X),

 write(' to '),

 write(Y),

 nl.

move(N,X,Y,Z):-

 N > 1,

 M is N - 1,

 move(M,X,Z,Y),

 move(1,X,Y,_),

 move(M,Z,Y,X).

% Define a predicate to solve the Towers of Hanoi puzzle

stoh(N) :-

 move(N,left,right,center).

OUTPUT:

?- stoh(3).

Move disk from left to right

Move disk from left to center

Move disk from right to center

Move disk from left to right

Move disk from center to left

Move disk from center to right

Move disk from left to right

true .

RESULT: Thus, the Program to solve Towers of Hanoi was executed Successfully

EX.NO: 14

CRYPT ARITHMETIC PROBLEM

AIM:

To Write a Prolog Program to solve Crypt Arithmetic Problem.

S E N D

M O R E

M O N E Y

PROGRAM:

alpha1(S, E, N, D, M, O, R, Y, [S, E, N, D, M, O, R, E, M, O, N, E, Y]) :-

between(1, 9, S),

between(0, 9, E),

E \= S,

between(0, 9, N),

N \= E, N \= S,

between(0, 9, D),

D \= N, D \= E, D \= S,

between(1, 9, M),

M \= D, M \= N, M \= E, M \= S,

between(0, 9, O),

O \= M, O \= D, O \= N, O \= E, O \= S,

between(0, 9, R),
 $R \neq O, R \neq M, R \neq D, R \neq N, R \neq E, R \neq S,$
 between(0, 9, Y),
 $Y \neq R, Y \neq O, Y \neq M, Y \neq D, Y \neq N, Y \neq E, Y \neq S,$
 Send is $S*1000 + E*100 + N*10 + D,$
 More is $M*1000 + O*100 + R*10 + E,$
 Money is $M*10000 + O*1000 + N*100 + E*10 + Y,$
 Money is Send + More.

OUTPUT:

?-alpha1(S, E, N, D, M, O, R, Y, Sol).

S = 9,

E = 5,

N = 6,

D = 7,

M = 1,

O = 0,

R = 8,

Y = 2,

Sol = [9, 5, 6, 7, 1, 0, 8, 5, 1|...];

RESULT: Thus, the Crypt Arithmetic Problem was solved by using Prolog